EECS 3311 — SOFTWARE DESIGN | STAGE 2 DELIVERABLE

Personal Finance AI Agent: Stage 2 Implementation Report

Author: Karthik Kumaran (220162491) | **Repository:** github.com/thickKar1/EECS3311-Finance-Agent

1. Stage 1 to Stage 2 Requirement Traceability

The Personal Finance AI Agent was implemented in Java 21 without external third-party framework dependencies, using the standard Java HTTP client, Swing GUI toolkit, and CSV-backed file persistence. All ten core functional requirements (F01–F10) defined in Stage 1 were delivered and verified through live natural-language agent routing.

ID	Feature Name	Implementing Classes	Stage 2 Implementation & Alignment
F01	Budget Management	`BudgetTool`, `BudgetTracker`, `AlertNotifier`	Sets user monthly spending caps, computes utilization percentage, and registers threshold observers.
F02	Transaction Entry	`TransactionTool`, `DataManager`	Validates positive amounts, records categorized expenses to CSV, and notifies budget observers.
F03	Spending Summary	`SummaryTool`, `DataManager`	Aggregates total expenditure and itemizes category-level breakdowns for the active user.
F04	Anomaly Detection	`AnomalyTool`, `DataManager`	Computes mean transaction size and flags expenses exceeding twice the user's average spend.
F05	Savings Goal Tracking	`GoalTool`, `DataManager`	Tracks deposits toward a target savings goal and reports completion percentage and remaining balance.
F06	Tax Estimator	`TaxTool`	Calculates estimated progressive income tax, effective tax rate, and net after-tax income.
F07	Debt Management	`DebtTool`	Computes monthly amortization payoff duration and total interest; rejects payments below monthly interest.
F08	Currency Conversion	`CurrencyTool`	Converts monetary amounts across supported currencies (such as USD to CAD) with formatted output.
F09	Investment Tracking	`InvestmentTool`, `DataManager`	Records portfolio contributions, displays asset allocation, and projects 1-year compounded value.
F10	Bill Reminders	`BillTool`, `DataManager`	Retrieves upcoming recurring bills with due-date countdowns and payment amounts.

2. Software Architecture & Design Patterns

The system separates user interface concerns, LLM orchestration, financial tool execution, and storage into modular Java classes governed by four GoF design patterns:

- **Facade Pattern (`ApplicationFacade`):** Exposes a single unified interface (`submitQuery` , `runGUI` , `runCLI`) to both the Java Swing desktop window and the command-line runner (`--mode=cli`), hiding internal agent orchestration, tool creation, and database operations.
- **Factory Method Pattern (`ToolFactory` & `FinanceTool`):** Decouples tool selection from concrete instantiation. When `FinanceAgent` resolves an intent label (e.g., `budget` , `tax` , `currency`), `ToolFactory` returns the corresponding `FinanceTool` implementation.
- **Observer Pattern (`BudgetTracker` , `BudgetObserver` , `AlertNotifier`):** Decouples transaction logging from threshold monitoring. Whenever a new expense updates cumulative spending in `BudgetTracker` , registered `BudgetObserver` instances are notified automatically to emit budget-exceeded warnings.
- **Singleton Pattern (`DataManager`):** Provides a single synchronized access point (`DataManager.getInstance()`) for reading and writing user transactions, budgets, savings goals, and portfolio balances in `finance_transactions.csv` .

3. Agentic Loop, LLM Integration & Fault Tolerance

Natural-language requests follow a two-stage agentic pipeline inside `FinanceAgent` and `LLMClient` , configured externally via environment variables loaded from `.env` (`LLM_API_KEY` and `LLM_MODEL` , modeled after `.env.example`):

- **Stage 1 — Intent & Argument Extraction:** `FinanceAgent` sends the user's prompt and available tool schemas to the OpenRouter Chat Completions API via `LLMClient` to identify the target tool name and structured arguments.
- **Stage 2 — Deterministic Tool Execution:** The selected `FinanceTool` executes validated Java financial logic and updates persistent CSV state, ensuring numerical calculations and boundary checks are never hallucinated by the language model.
- **Stage 3 — Response Synthesis & Rate-Limit Fallback:** The structured tool output is passed back to `LLMClient` to compose a natural-language response. If the upstream free-tier API returns `HTTP 429` (rate limit exceeded) or a network interruption occurs, `LLMClient` recovers gracefully using deterministic intent matching and formatted tool output so execution never crashes.

4. Demonstration Video & Feature Verification

> **Screen Recording Link:**
> https://drive.google.com/file/d/143mHo5AnIn51OH96z8g8egOXr_S5bnVI/view?usp=sharing
> **Verification Scope:** Live execution in Swing GUI and CLI (`--mode=cli`) modes with `.env` credentials kept private off-screen.

Mode	Feature / Test Case	Prompt / Action Demonstrated	Verified System Behavior
Swing GUI	F01: Budget Management	`Set my budget limit to 1000`	Sets limit to $1,000.00 (0.0% used).
Swing GUI	F02: Transaction & Alert	`Log $250 for Groceries` / `Log $900 for Electronics`	Logs expenses and triggers Observer budget alert.
Swing GUI	F02: Invalid Input Handling	`Log -50 for Food`	Rejects non-positive transaction amount cleanly.
Swing GUI	F03: Spending Summary	`Show my spending summary`	Reports $1,150.00 total across categories.
Swing GUI	F04: Anomaly Detection	`Check for unusual spending`	Flags Electronics ($900.00) exceeding 2x average.
Swing GUI	F05: Savings Goal	`Deposit 500 into my savings goal`	Updates savings progress to $3,000 / $5,000 (60.0%).
Swing GUI	F06: Tax Estimator	`Estimate tax on 75000 income`	Computes $12,250.00 tax (16.3% effective rate).
Swing GUI	F07: Debt & Error Check	`Pay 250...` / `Pay 10 per month toward my debt`	Computes 7-month payoff; rejects sub-interest payment.
Swing GUI	F08: Currency Conversion	`Convert 250 USD to CAD`	Converts 250.00 USD to 345.00 CAD.
Swing GUI	F09: Investment Tracking	`Invest 2000 into my portfolio`	Updates balance to $12,000 ($12,840 1-yr projection).
GUI & CLI	F10: Bill Reminders & CLI	`Show my upcoming bills` + `--mode=cli`	Lists Internet ($80) & Phone ($55); shows CLI routing.

5. AI Tool Usage & Verification

Generative AI assistance (Google Gemini) was used during Stage 2 to scaffold boilerplate Java tool classes, refine OpenRouter HTTP request/error-handling logic in `LLMClient.java`, structure test verification prompts, and draft documentation templates. All generated code was compiled locally with `javac`, tested end-to-end across GUI and CLI modes, and verified against Stage 1 functional requirements prior to submission.